ENGINEERING PROJECT RECORD

DevMetrics

Backend Development Documentation — Part 1

GitHub Integration | PostgreSQL + Prisma | Authentication & JWT

Developer Productivity Intelligence Platform

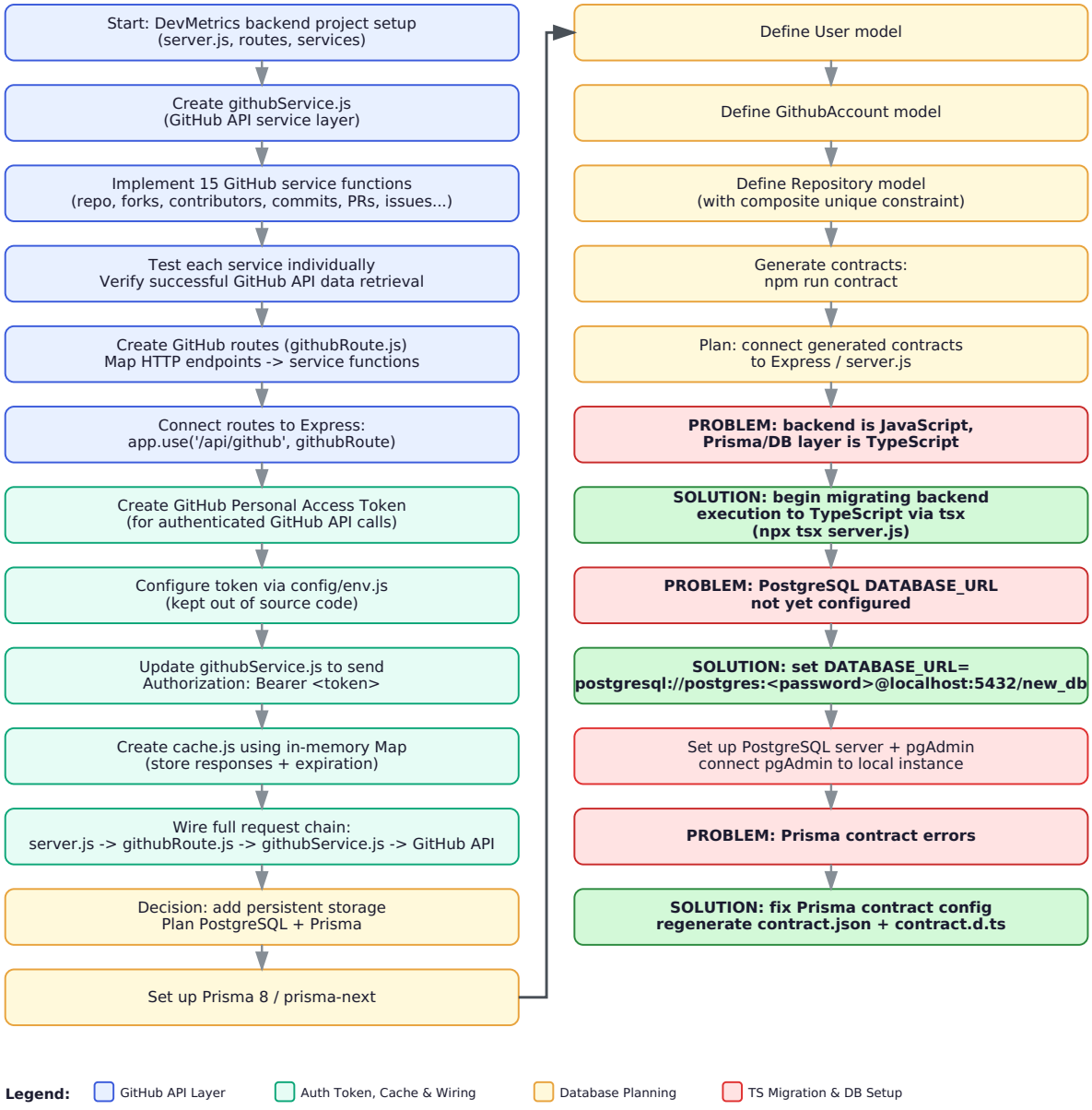
Backend implementation record: GitHub service layer through authentication middleware

Status snapshot: authentication middleware in progress

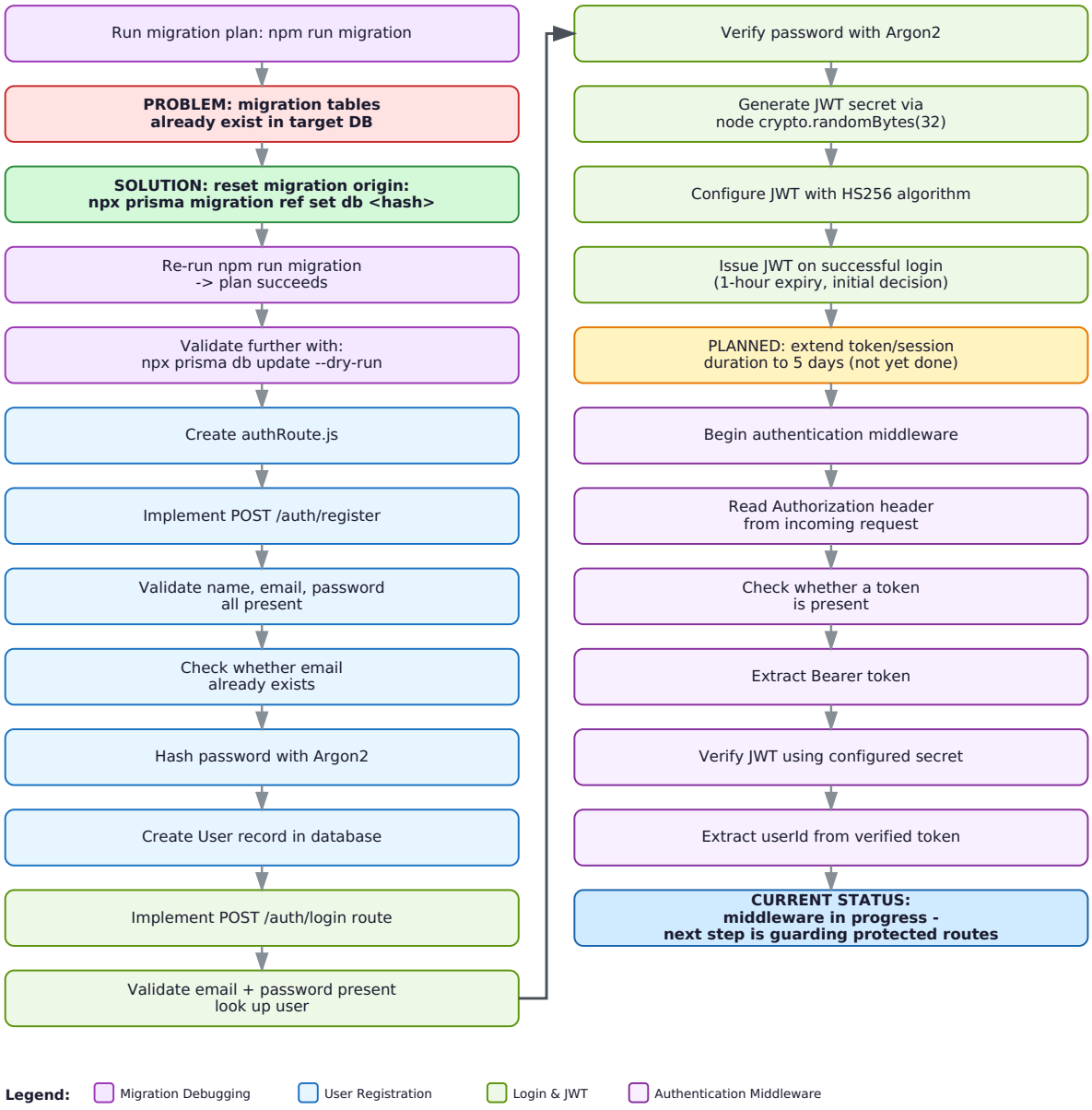
Flowchart 1 — Development Journey

Chronological view of the DevMetrics backend build, from initial GitHub service creation through to the current authentication middleware work. Problem boxes (red) are paired with the solution that resolved them (green); planned/future work is marked yellow; the current status is marked blue. Shown across two pages for readability.

Flowchart 1a — DevMetrics Development Journey (Steps 1–25)



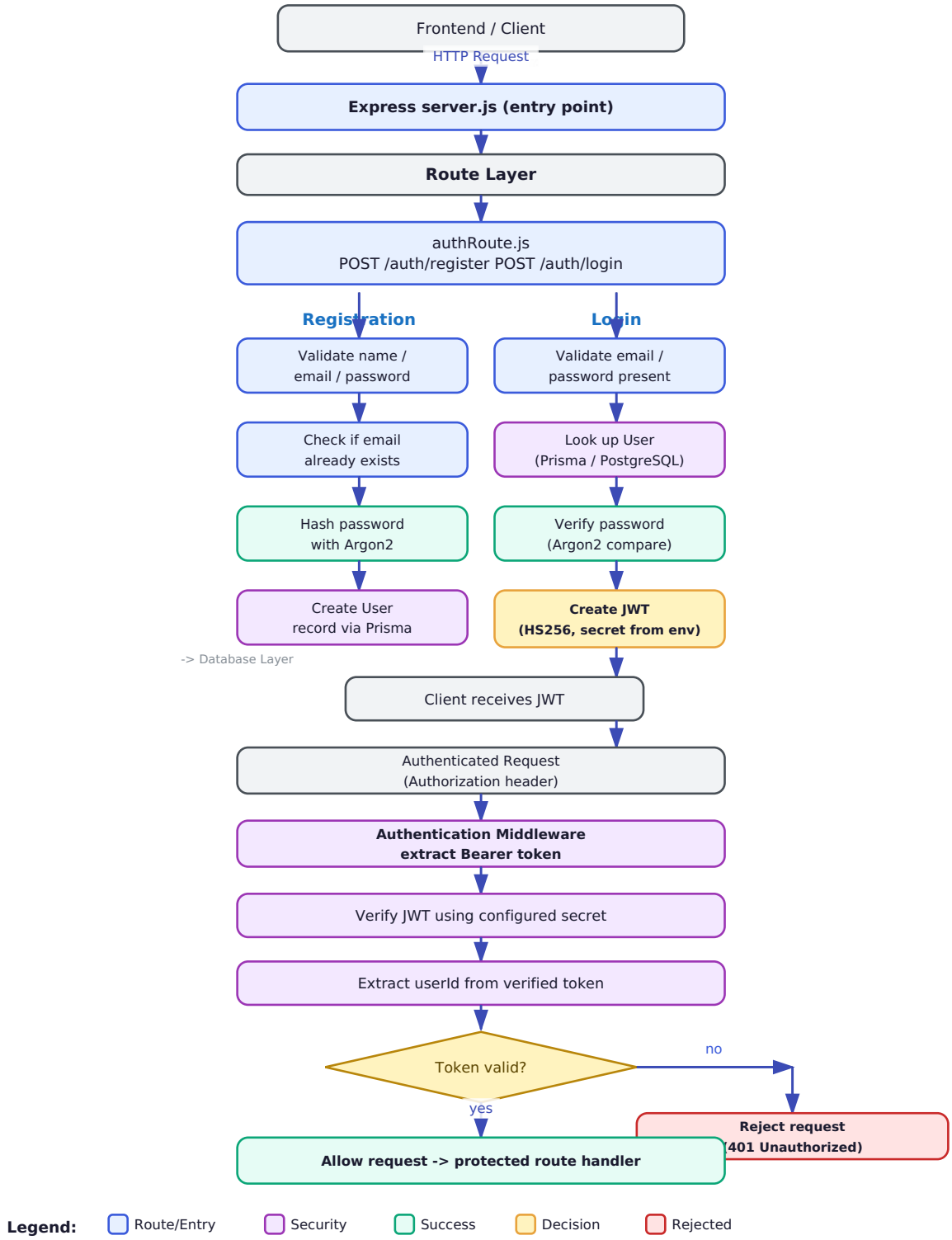
Flowchart 1b — DevMetrics Development Journey (Steps 26–50)



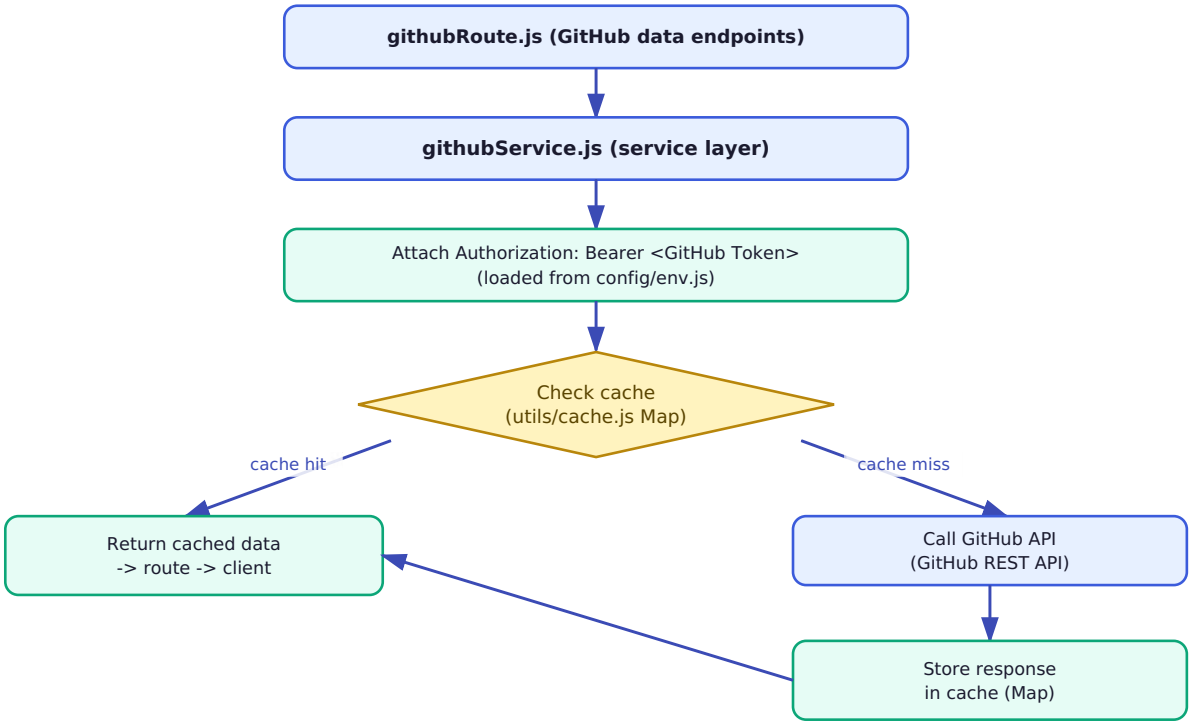
Flowchart 2 — Complete Program / Request Flow

Architectural view of the current backend: how a request actually moves through the system today, across the routing layer, the authentication path, the GitHub data path (including caching), and the database layer. Shown across two pages.

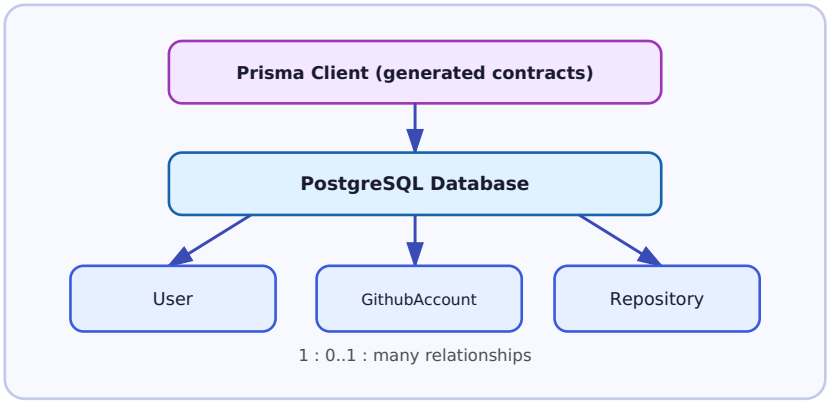
Flowchart 2a — Request Flow: Routing & Authentication



Flowchart 2b — Request Flow: GitHub Data & Database



Database Layer (shared by auth routes)



Auth routes reach this same Database Layer via Prisma
(see Flowchart 2a — registration & login paths).

Legend: Route/Service · Security/Data · Success/Storage · Decision

1. Project Overview

DevMetrics is a Developer Productivity Intelligence Platform. The backend documented here is the data and logic layer that will eventually power analytics on engineering activity — it is responsible for retrieving raw activity data from GitHub, persisting application data (users, linked GitHub accounts, tracked repositories), and authenticating the users of the platform.

The backend is intended to support analytics such as:

- Commit activity and history
- Pull request throughput and review/merge cycles
- Code review participation
- Issue tracking and resolution timelines
- Repository-level metadata (languages, contributors, branches)
- Developer and team-level productivity trends

To support this, the backend combines two concerns: (1) a **GitHub integration layer** that pulls activity data directly from the GitHub REST API, and (2) an **application data layer** (PostgreSQL via Prisma) that stores DevMetrics' own users, their linked GitHub accounts, and the repositories being tracked. Section 2 documents the initial architecture that was built to support this.

2. Initial Backend Architecture

The backend was initially built as a JavaScript / Express application with a clear separation between routing, service logic, configuration, and caching. The table below documents each core file's role at this stage of the project.

File	Layer	Responsibility
`server.js`	Entry point	Main Express application; boots the HTTP server and mounts the route layer.
`githubRoute.js`	Routing	HTTP routing layer for GitHub-related requests; maps incoming requests to the corresponding service functions.
`githubService.js`	Service	Contains the logic that communicates directly with GitHub's API (the individual `fetch*` functions documented in Section 3).
`env.js`	Configuration	Exposes environment/configuration values — most importantly the GitHub Personal Access Token — to the rest of the backend without hard-coding secrets in source files.
`cache.js`	Utility	In-memory caching utility (built on a JavaScript `Map`) used to reduce repeated GitHub API calls.

> The frontend/client is expected to call the DevMetrics backend rather than GitHub directly — the backend acts as an authenticated, cached intermediary between the DevMetrics frontend and GitHub's API, and will also serve DevMetrics' own authentication and application data.

3. GitHub Service Implementation

The first phase of implementation focused on building the GitHub service layer: a set of functions in `githubService.js` responsible for retrieving GitHub activity data. Each function below was implemented and individually tested to confirm it could successfully retrieve data from the GitHub API before the route layer was built on top of it.

Service Function	Purpose in DevMetrics
`fetchGithubRepo`	Retrieves core repository metadata, the base data point the rest of the analytics layer builds on.
`fetchForks`	Retrieves a repository's forks, supporting reach/adoption-style metrics.
`fetchContributors`	Retrieves the list of contributors, used for team/developer-level activity views.
`fetchCommits`	Retrieves commit history — a core input for commit-frequency and activity analytics.
`fetchPullRequests`	Retrieves pull requests, feeding PR throughput and review-cycle analytics.
`fetchIssues`	Retrieves issues, feeding issue-tracking and resolution-time analytics.
`fetchLanguages`	Retrieves the language breakdown of a repository, used for tech-stack context.
`fetchCollaborators`	Retrieves collaborators with repository access, supporting team-composition views.
`fetchBranches`	Retrieves branch data, supporting workflow/branching-pattern analysis.
`fetchIndividualCommit`	Retrieves details for a single commit, used for drill-down views.
`fetchIndividualPR`	Retrieves details for a single pull request, used for drill-down views.
`fetchPRReviews`	Retrieves reviews on a pull request, feeding review-participation and merge-cycle analytics.
`fetchPRComments`	Retrieves PR discussion comments, supporting collaboration analytics.
`fetchIndividualIssue`	Retrieves details for a single issue, used for drill-down views.
`fetchIssueTimeline`	Retrieves the event timeline of an issue, supporting resolution-time analytics.

COMPLETED All fifteen functions above sit in the service layer of the architecture (see Flowchart 2) and were tested individually to verify successful data retrieval from the GitHub API. Exact endpoint URLs are not reproduced here, as they are not specified in the source notes.

4. GitHub Routes

With the service layer in place, a routing layer was introduced between the client and the services. This separation exists so that HTTP concerns (paths, request/response handling) stay independent from the logic that talks to GitHub:

```
Frontend / Client
    ↓
Express Route  (githubRoute.js)
    ↓
githubService.js
    ↓
GitHub API
```

The route layer's only responsibility is to receive an HTTP request and delegate to the correct service function; the service layer's only responsibility is to know how to talk to GitHub. This keeps GitHub-specific logic out of the routing code and makes each service function independently testable, which is how Section 3's services were verified before routing existed.

5. Express Server Connection

The GitHub route module was connected to the Express application with:

```
app.use('/api/github', githubRoute);
```

This line mounts the entire `githubRoute` router under the `/api/github` path prefix. Practically, this means any request beginning with `/api/github/...` is forwarded by Express to the GitHub router, which then matches the remaining part of the path to a specific route handler and invokes the corresponding function in `githubService.js`. This is the mechanism shown as "Route Layer" in Flowchart 2.

6. GitHub Personal Access Token

COMPLETED

Early GitHub API requests were unauthenticated. Unauthenticated requests against the GitHub API are subject to much lower rate limits and cannot access certain data, so authentication was introduced:

1. A GitHub Personal Access Token was created.
2. The token was stored through the environment configuration layer, `env.js`, rather than being written directly into `githubService.js`.
3. `env.js` exposes the token to the service layer, which attaches it to outgoing GitHub API requests as:

```
Authorization: Bearer <GitHub Token>
```

> Security principle: secrets (tokens, passwords, JWT signing keys) are kept out of source code and out of this document. Wherever a real credential would appear in the underlying project, it is represented here only as a placeholder.

7. Cache System

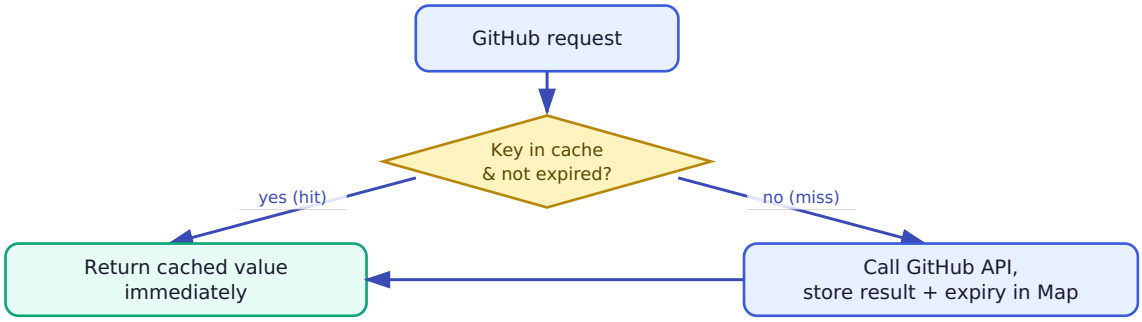
COMPLETED

`cache.js` introduces a lightweight in-memory cache built on a JavaScript `Map`, sitting between `githubService.js` and the GitHub API. Its purpose is to avoid re-requesting the same data from GitHub on every call, which both reduces latency and helps stay under GitHub's API rate limits.

The cache supports two operations relevant to the request flow:

- **Set** — store a GitHub API response in the Map along with an expiration time.
- **Get** — retrieve a previously cached response if it exists and has not expired.

Cache Lookup Flow (utils/cache.js — in-memory Map)



Caching reduces repeated GitHub API calls and helps avoid GitHub's rate limits.

8. Database Introduction

Once the GitHub integration layer was functional (services tested, routes wired, token-based auth and caching in place — see Flowchart 1), the project moved from a GitHub-API-only backend toward one with persistent application data. This required a real database rather than in-memory state.

The decision was made to introduce:

- **PostgreSQL** as the relational database.
- **Prisma** as the ORM / schema and migration layer, specifically **Prisma 8** using the `prisma-next` generation path, as reflected directly in the project's schema comments.
- Three initial models: **User**, **GithubAccount**, and **Repository** — covering DevMetrics' own accounts, each account's linked GitHub identity, and the repositories tracked under that identity.

9. Database Schema

The schema below is documented as defined in the project's Prisma files (Prisma 8 / `prisma-next`).

User

```
model User {
  id            Int      @id @default(autoincrement())
  name          String
  email         String   @unique
  passwordHash  String

  githubAccount GithubAccount?

  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt
}
```

- `id` is the primary key and auto-increments with each new user.
- `email` is marked unique, preventing duplicate accounts on the same email address.
- The password is stored as `passwordHash` — never as plaintext (see Section 13).
- `githubAccount` is an optional one-to-one relation: a user may or may not have linked a GitHub account yet.
- `createdAt` / `updatedAt` are standard audit timestamps.

GithubAccount

```
model GithubAccount {
  id             Int      @id @default(autoincrement())
  githubUsername String
  encyptedToken  String

  userId         Int      @unique
  user           User     @relation(fields: [userId], references: [id])

  repositories   Repository[]

  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt
}
```

- Belongs to exactly one `User` via the unique `userId` foreign key (one-to-one from the User side).
- Has many `Repository` records — the repositories tracked under this GitHub identity.

> **Naming note:** the field is documented in the schema as `encyptedToken`. This is preserved here exactly as written because it reflects the actual field name in the project, but it is flagged as a likely spelling issue — it most probably should read `encryptedToken`. This should be corrected before the field is relied upon elsewhere (e.g. in generated contracts or client code), since renaming later would be a breaking schema change.

Repository

```
model Repository {
  id              Int      @id @default(autoincrement())
  githubRepoId    Int
  repoName        String
  fullName        String

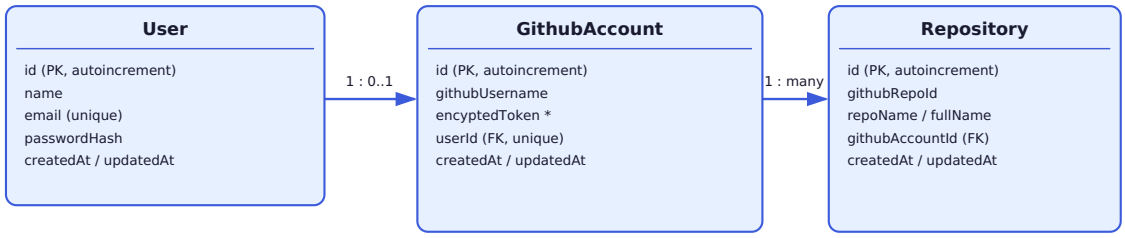
  githubAccountId Int
  githubAccount   GithubAccount @relation(fields: [githubAccountId], references: [id])

  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@unique([githubAccountId, githubRepoId])
}
```

Each repository belongs to one `GithubAccount`. The composite unique constraint `@@unique([githubAccountId, githubRepoId])` ensures the same GitHub repository cannot be inserted twice under the same GitHub account — `githubRepoId` alone isn't unique across accounts, but the pairing of account and repo ID must be.

Entity-Relationship Diagram — Core Data Models



@@unique([githubAccountId, githubRepoId]) prevents the same GitHub repo from being duplicated under the same GitHub account.

* "encyptedToken" is documented as written in the schema notes — flagged here as a likely spelling issue (encryptedToken) to fix later.

10. Contract Generation

COMPLETED

After the schema was defined, contracts were generated with:

```
npm run contract
```

Based on the project's architecture, this step generates typed artifacts from the Prisma schema — `contract.json` and `contract.d.ts` — that describe the shape of the database's data models. The stated intent was to connect these generated contracts into the Express/server layer so the backend has a typed, generated reference for its data shapes rather than hand-maintained types. The precise mechanism by which the Express layer consumes these contracts is not detailed in the source notes and is not claimed here beyond what was stated.

11. Problems Encountered During Database Integration

This section documents every issue encountered while integrating PostgreSQL and Prisma, using a Problem → Solution format. Nothing here is smoothed over — these are the actual blockers hit during this phase of the build.

PROBLEM 1 — JAVASCRIPT BACKEND VS. TYPESCRIPT DATABASE LAYER

The backend was originally written in JavaScript, but the Prisma / database setup involved TypeScript. This raised a compatibility question: could the existing JavaScript backend correctly consume a TypeScript-based database layer and its generated types?

SOLUTION

The backend began migrating its execution model toward TypeScript, running the server through `tsx` instead of plain Node:

```
npx tsx server.js
```

This allows the existing server code to execute in a way that is compatible with the TypeScript-based Prisma layer. This is documented here strictly as the command that was used — the full scope of the JS→TS migration is ongoing rather than complete.

PROBLEM 2 — POSTGRESQL DATABASE_URL CONFIGURATION

Prisma requires a `DATABASE_URL` connection string to reach PostgreSQL, which had not yet been configured.

SOLUTION

A connection string was defined following this structure:

```
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/new_db"
```

`YOUR_PASSWORD` is a placeholder only — it stands in for wherever a real local PostgreSQL password would be supplied via environment configuration. No actual credential is documented here.

PROBLEM 3 — POSTGRESQL / PGADMIN SETUP

Beyond the connection string itself, the local PostgreSQL instance needed to be running and observable, and Express needed to be confirmed as actually reaching Prisma/the database.

SOLUTION

- PostgreSQL server brought up and running locally.
- pgAdmin connected to the running PostgreSQL instance for inspection.
- `DATABASE_URL` configured (Problem 2).
- Express confirmed connected through Prisma to the database.

PROBLEM 4 — PRISMA CONTRACT PROBLEMS

An issue occurred with the Prisma contract generation/configuration.

SOLUTION

The Prisma contract configuration was fixed, after which both generated artifacts were produced successfully: `contract.json` and `contract.d.ts`.

PROBLEM 5 — EXISTING MIGRATION TABLES

Running the migration plan surfaced an error indicating that migration tables already existed in the target database, blocking the plan from proceeding cleanly:

```
npm run migration
```

SOLUTION

The database's migration origin/reference was explicitly reset to a known migration:

```
npx prisma migration ref set db 1e8412e162dbbe69f4bb3bf8d07f0280ae67eaab15c34dcf201e67468315428d
```

After resetting the migration reference, the migration command was re-run:

```
npm run migration
```

The migration plan succeeded on this second run. The long identifier above is treated as a migration/database reference from the project's own history, not as a secret value.

PROBLEM 6 — ADDITIONAL DATABASE VALIDATION

Even after the migration plan succeeded, further validation of pending database changes was needed before committing to them.

SOLUTION

A dry-run check was used to inspect proposed changes without applying them:

```
npx prisma db update --dry-run
```

This command is documented here as a validation step only; no specific result of this dry-run is recorded in the source notes, so none is claimed.

12. Database Status Checkpoint (Historical)

> This checklist captures the project's state **at one specific point in time** — immediately after the database/Prisma problems in Section 11 were resolved, and before registration, login, or JWT work began. It is a historical snapshot, not the current status. The current status is tracked separately in Section 21.

- ☑ PostgreSQL running
- ☑ pgAdmin connected
- ☑ DATABASE_URL configured
- ☑ Express connected to Prisma/database
- ☑ Prisma contract fixed
- ☑ contract.json generated
- ☑ contract.d.ts generated
- ☐ Migration (fully finalized)
- ☐ User table (in active use)
- ☐ Registration
- ☐ Password hashing
- ☐ Login
- ☐ JWT
- ☐ GitHub OAuth / token handling (user-linked)

From this checkpoint, work proceeded directly into user registration (Section 13), login and JWT issuance (Sections 14–15), and authentication middleware (Section 16) — each of which subsequently reached the state documented in those sections.

13. User Registration

COMPLETED

`authRoute.js` was created inside the routes folder to hold DevMetrics' authentication endpoints, starting with `POST /auth/register`.

```
Request
    ↓
Check name present
    ↓
Check email present
    ↓
Check password present
    ↓
Check whether email is already registered
    ↓
Hash password using Argon2
    ↓
Create User record
    ↓
Store passwordHash (never the raw password)
```

The registration handler validates that all three required fields — `name`, `email`, and `password` — are present before doing any further work. It then checks whether the submitted email is already registered, to enforce the uniqueness constraint defined on `User.email` (Section 9) at the application level as well as the database level.

> Passwords are never stored directly. The plaintext password is only ever used transiently to compute an Argon2 hash; only `passwordHash` is persisted to the `User` table.

14. Login

COMPLETED

```
Login Request
    ↓
Check email and password present
    ↓
Find user by email
    ↓
Verify password (Argon2 compare against passwordHash)
    ↓
Authenticate user
    ↓
Generate JWT
    ↓
Return token to client
```

The login handler first checks that both email and password were supplied in the request; if either is missing, the request is rejected before a database lookup is attempted. If the fields are present, the user is looked up by email and the submitted password is verified against the stored `passwordHash` using Argon2. Only once verification succeeds does the handler proceed to issue a JWT.

15. JWT Authentication

COMPLETED (secret generation, algorithm, issuance) **PLANNED** (duration extension)

Secret generation and algorithm

The JWT signing secret was generated using Node's built-in `crypto` module:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

The resulting 32-byte hex value is used as the JWT secret. **HS256** was chosen as the signing algorithm because it is a symmetric algorithm requiring only a single shared secret (as opposed to an asymmetric algorithm like RS256, which needs a public/private key pair) — appropriate here since only the DevMetrics backend itself needs to verify its own tokens.

> The generated secret should be stored securely (e.g. via environment configuration, consistent with how the GitHub token is handled in `env.js`) rather than hard-coded into source files.

Token duration

The JWT was initially issued with a **1-hour** expiry during login. This was judged too short for practical use of the application, and a change to a **5-day** duration was

planned.

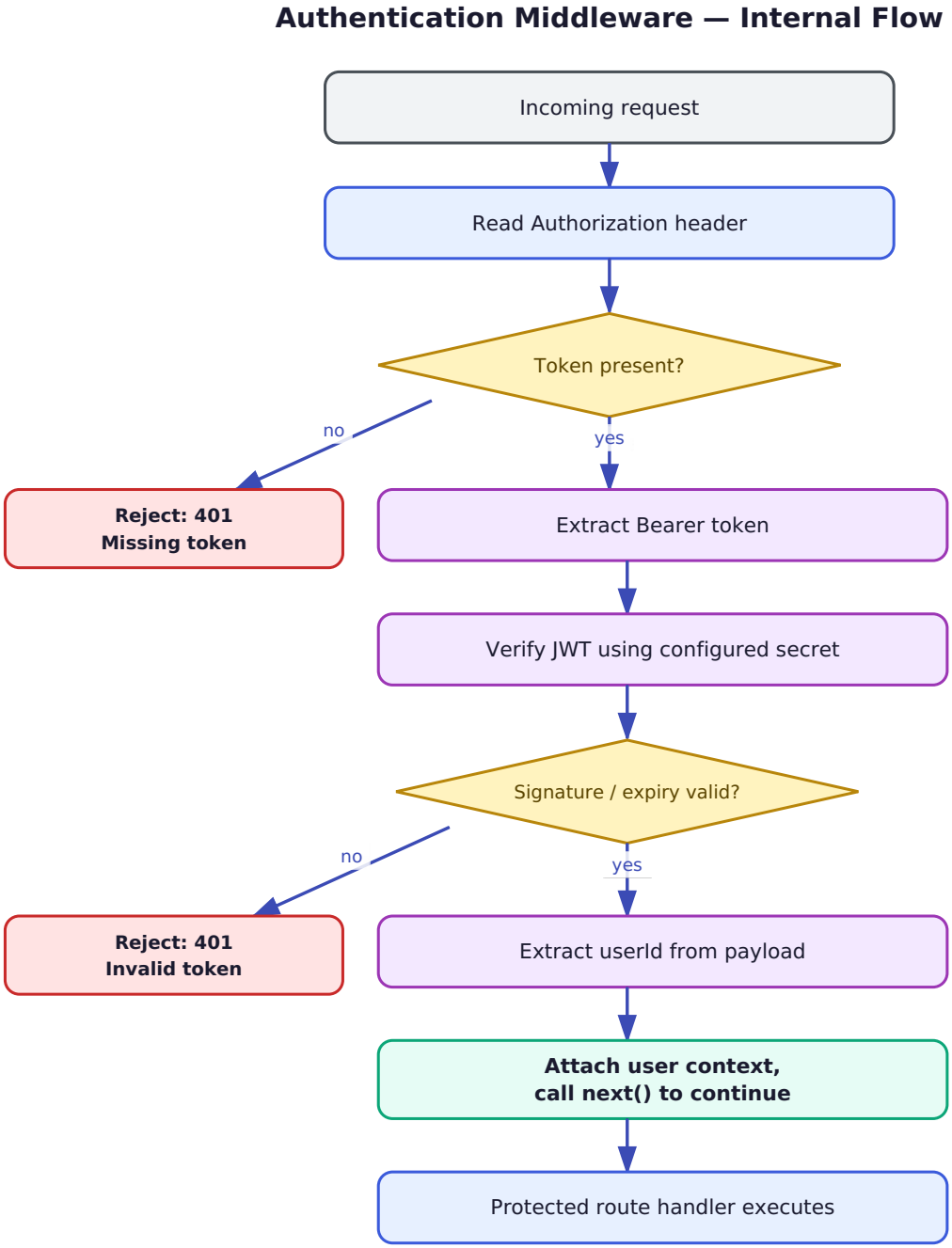
> **PLANNED / FUTURE** The 5-day session duration had **not yet been implemented** as of this documentation snapshot — the token is still issued with a 1-hour expiry today. This is recorded as a planned change, not a completed one.

16. Authentication Middleware

IN PROGRESS

Work is currently underway on the Express middleware that will protect routes requiring an authenticated user. Its responsibilities, as scoped so far, are:

1. Read the incoming request.
2. Read the `Authorization` header.
3. Check whether a token is present in that header.
4. Extract the Bearer token from the header value.
5. Verify the JWT using the configured JWT secret.
6. Extract `userId` from the verified token's payload.
7. Allow the request to continue to its handler if verification succeeds; reject it otherwise.



Current status: middleware verifies tokens and extracts userId; wiring it onto specific protected GitHub routes is the next step.

17. Complete Error & Solution Log

Problem	Where / When	Cause	Investigation / Command	Solution	Status
Unauthenticated GitHub requests	GitHub service layer, early phase	No token attached to GitHub API calls	Manual review of request behavior	Created GitHub PAT; loaded via env.js; added `Authorization: Bearer` header	RESOLVED
JS backend vs. TS database layer	Start of DB integration	Backend written in JavaScript; Prisma/DB layer in TypeScript	Manual compatibility check	Began running backend via `npx tsx server.js`	MIGRATION ONGOING
PostgreSQL connection not configured	DB setup	`DATABASE_URL` missing	Prisma connection error	Set `DATABASE_URL="postgresql://postgres:<password>@localhost:5432/new_db"`	RESOLVED
PostgreSQL / pgAdmin not verified	DB setup	Local instance and inspection tool not yet confirmed working	Manual verification	PostgreSQL run locally; pgAdmin connected; Express-Prisma link confirmed	RESOLVED
Prisma contract errors	Contract generation	Contract configuration issue	Reviewed Prisma contract setup	Fixed configuration; regenerated `contract.json` and `contract.d.ts`	RESOLVED
Migration tables already exist	Migration planning	Target DB already contained migration tables from a prior state	`npm run migration`	`npx prisma migration ref set db <hash>`, then re-ran `npm run migration`	RESOLVED
Further DB change validation needed	Post-migration	Wanted to inspect changes before applying	`npx prisma db update --dry-run`	Used as a validation/inspection step	VALIDATION STEP; NO FURTHER RESULT RECORDED
Registration field validation	authRoute.js — /auth/register	Needed to guarantee name/email/password all present before proceeding	Manual route design	Added presence checks + duplicate-email check before hashing	RESOLVED
Password storage risk	Registration	Storing raw passwords is unsafe	Manual design decision	Hash with Argon2; store only `passwordHash`	RESOLVED
JWT secret management	Login implementation	Needed a signing secret for JWTs	`node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`	Generated secret; configured HS256	RESOLVED
Token duration too short	Login / JWT issuance	1-hour expiry impractical for normal usage	Manual review of session length	Planned change to 5-day duration	PLANNED
Route-level authentication enforcement	Middleware	No mechanism yet to guard protected routes	In-progress middleware design	Building header check → token extraction → JWT verification → userId extraction	IN PROGRESS

18. Commands & Tools Used

Command	Purpose
`npm run contract`	Generate Prisma-based contracts (`contract.json` , `contract.d.ts`)
`npm run migration`	Run/plan the Prisma database migration
`npx prisma migration ref set db <hash>`	Reset the migration origin/reference after discovering pre-existing migration tables
`npx prisma db update --dry-run`	Validate pending database changes without applying them
`npx tsx server.js`	Run the Express server through `tsx` to bridge the JavaScript backend with the TypeScript database layer
`node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`	Generate a cryptographically random JWT signing secret

Only commands explicitly present in the source material are listed. No package-installation commands are documented here, as the exact install commands/packages were not specified in the notes.

19. File-by-File Reference

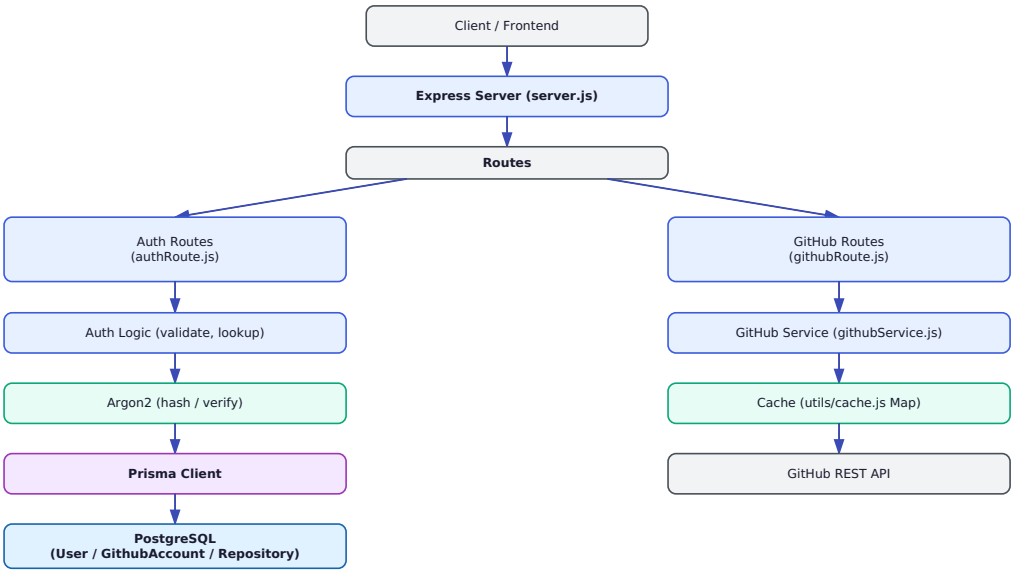
File	Layer	Responsibility	Connects To
`server.js`	Entry point	Boots Express, mounts routes	githubRoute.js, authRoute.js
`routes/githubRoute.js`	Routing	Maps GitHub-related HTTP endpoints to service functions	server.js → githubService.js
`routes/authRoute.js`	Routing	Handles `/auth/register` and `/auth/login`	server.js → Argon2, Prisma, JWT logic
`services/githubService.js`	Service	Implements all GitHub API fetch functions; attaches Bearer token	githubRoute.js → GitHub API, cache.js
`config/env.js`	Configuration	Exposes GitHub token and other environment/config values	githubService.js
`utils/cache.js`	Utility	In-memory Map-based cache with expiration	githubService.js
Prisma schema (models)	Data	Defines User, GithubAccount, Repository models	Prisma Client → PostgreSQL
`contract.json`	Generated	Generated data-contract description from the Prisma schema	Backend / Express layer
`contract.d.ts`	Generated	TypeScript type definitions generated alongside contract.json	Backend / Express layer
`package.json`	Config	Declares scripts including `contract` and `migration`	npm / npx tooling
`.env`	Configuration	Holds `DATABASE_URL` , GitHub token, JWT secret	env.js, Prisma, JWT logic

Data flows top-down through these layers: the entry point mounts routes, routes call services, services call external APIs or the database layer, and configuration/utility files (env, cache) are consumed horizontally by the service layer.

20. Current Architecture

The diagram below is a compact snapshot of the backend as currently implemented, combining the routing, service, security, and data layers documented above.

Current DevMetrics Backend Architecture (Snapshot)



Authentication Sequence



Legend: Blue = routing / core flow Purple = security & data-access layer Green = supporting utility Grey = infra / external
Orange = token issuance. This reflects the backend as of the current documented state (auth middleware in progress).

21. Current Project Status & Next Steps

Completed

- ☑ GitHub service layer (15 functions) implemented and individually tested
- ☑ GitHub routes created and connected to Express (`/api/github`)
- ☑ GitHub Personal Access Token created and wired through env.js
- ☑ Bearer-token authentication added to GitHub API requests
- ☑ In-memory cache (Map-based) implemented for GitHub responses
- ☑ PostgreSQL + Prisma set up; User, GithubAccount, Repository models defined
- ☑ Contracts generated (contract.json, contract.d.ts) after fixing Prisma contract issues
- ☑ Migration conflict resolved; migration plan runs successfully
- ☑ User registration implemented (validation, duplicate-email check, Argon2 hashing, user creation)
- ☑ Login implemented (validation, password verification, JWT issuance)
- ☑ JWT secret generated; HS256 configured; 1-hour token issuance working

In Progress

- ☐ Authentication middleware (header check, token extraction, JWT verification, userId extraction implemented; wiring onto protected routes is the remaining step)
- ☐ Full JavaScript → TypeScript backend migration (currently bridged via `tsx`)

Planned / Next

- ☐ Extend JWT session duration from 1 hour to 5 days
- ☐ Apply authentication middleware to protect GitHub data routes
- ☐ GitHub OAuth / per-user GitHub token handling (linking a DevMetrics user to their own GitHub account via `GithubAccount`)
- ☐ Build productivity analytics features on top of the now-authenticated, cached GitHub data pipeline

Documentation Part 1 — Summary

What Was Built

- Full GitHub service layer (15 tested functions)
- GitHub route layer wired into Express
- Token-authenticated GitHub API access
- In-memory response cache
- User registration and login endpoints
- JWT issuance (HS256)
- Authentication middleware (in progress)

What Was Integrated

- PostgreSQL database via Prisma 8 (prisma-next)
- User, GithubAccount, Repository data models
- Generated Prisma contracts (contract.json / .d.ts)
- Argon2 password hashing into the registration/login flow
- tsx-based execution bridging JS backend and TS database layer

Major Problems Solved

- JavaScript/TypeScript backend-database compatibility
- Missing PostgreSQL connection configuration
- Prisma contract generation errors
- Pre-existing migration tables blocking the migration plan

What Remains

- Finish and apply authentication middleware to protected routes
- Extend JWT duration to 5 days
- Implement GitHub OAuth / per-user token linking
- Complete the JS → TS backend migration
- Build analytics features on the underlying data pipeline

Current authentication architecture: registration and login issue Argon2-verified, HS256-signed JWTs; an in-progress middleware layer verifies these tokens and extracts `userId` for use by protected routes.

Current database architecture: PostgreSQL accessed through Prisma, with `User`, `GithubAccount`, and `Repository` models connected via one-to-one and one-to-many relationships, plus a composite uniqueness constraint preventing duplicate repository tracking.

This document reflects the DevMetrics backend as of the state described in the source development notes. It distinguishes verified implementation from historical checkpoints and planned/future work throughout, per the categorization used in each section.